Data Analysis: Airport Traffic, Monte Carlo Numerical Integration, and Generative AI

Daniel Carney

2026-04-16

Table of contents

1 Busiest Airports Analysis **2**

1.1 Data Collection and Preparation . . . 2

1.2 Passenger Traffic Table . . . 2

1.3 Passenger Traffic Visualization . . . 3

2 Monte Carlo Numerical Integration **4**

2.1 The Weibull Distribution Function . . . 4

2.2 Monte Carlo Methodology . . . 4

2.3 Results and Visualization . . . 4

3 Planning and Prompting GenAI Tools **6**

3.1 Data Context and Tidying Plan . . . 6

3.2 Data Context and Tidying Plan . . . 6

3.3 Plan Informed-Response . . . 6

3.4 Generic Response . . . 10

3.5 Comparative Analysis . . . 12

4 Reflection **12**

5 Appendix: Code Appendix **13**

5.1 Busiest Airports: Data Scraping and Cleaning . . . 13

5.2 Busiest Airports: Table Generation . . . 14

5.3 Busiest Airports: Plot Generation . . . 14

5.4 Monte Carlo Integration: Function Definition . . . 15

5.5 Monte Carlo Integration: Simulation and Visualization . . . 16

This document presents four sections: the first two sections reference work completed in previous homework assignments, examining global airport traffic patterns and demonstrating Monte Carlo numerical integration.

The third section explores the planning process for using generative AI tools in data analysis, and the final section offers a reflection on concepts learned throughout the course. All underlying code is contained in the Code Appendix at the end of this document.

1 Busiest Airports Analysis

1.1 Data Collection and Preparation

The data for this analysis was scraped from Wikipedia's "List of Busiest Airports by Passenger Traffic" page, which publishes annual rankings of the world's busiest airports based on total passenger amounts Using the `rvest` package in R, six consecutive annual tables (2020 through 2025) were programmatically scraped from the webpage. This used the `html_elements()` and `html_table()` functions to extract tabular data directly from the HTML source.

Six airports were selected for focused analysis: Hartsfield-Jackson Atlanta International (ATL), Dallas/Fort Worth International (DFW), Denver International (DEN), Los Angeles Inter

After scraping, the data required cleaning steps. Column names were standardized using the `clean_names()` function from the `janitor` package to ensure consistency across all six annual tables. The `totalpassengers` column contained reference annotations in brackets (e.g., `[1]`) that were stripped using a regular expression, after which the values were converted from character strings to numeric. Missing values in the `rankchange` column, which appeared when an airport's rank remained unchanged from the prior year, were replaced with zero. Finally, the six annual tables were combined into a single data frame and filtered to retain only the six airports of interest.

1.2 Passenger Traffic Table

Table 1: Rank and Total Passengers of Six Major International Airports (2020–2

Year	Rank	Code	Airport	Location	Co
2021	2	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	Unite
2021	4	DFW/KDFW	Dallas Fort Worth International Airport	Dallas-Fort Worth, Texas	Unite
2021	8	DEN/KDEN	Denver International Airport	Denver, Colorado	Unite
2021	15	LAX/KLAX	Los Angeles International Airport	Los Angeles, California	Unite
2021	19	DXB/OMDB	Dubai International Airport	Garhoud, Dubai	United Ar
2021	37	FRA/EDDF	Frankfurt Airport	Frankfurt, Hesse	Ger
2022	1	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	Unite
2022	2	DFW/KDFW	Dallas Fort Worth International Airport	Dallas-Fort Worth, Texas	Unite
2022	3	DEN/KDEN	Denver International Airport	Denver, Colorado	Unite
2022	5	LAX/KLAX	Los Angeles International Airport	Los Angeles, California	Unite
2022	28	DXB/OMDB	Dubai International Airport	Garhoud, Dubai	United Ar
2022	39	FRA/EDDF	Frankfurt Airport	Frankfurt, Hesse	Ger
2023	1	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	Unite
2023	2	DFW/KDFW	Dallas Fort Worth International Airport	Dallas–Fort Worth, Texas	Unite
2023	3	DEN/KDEN	Denver International Airport	Denver, Colorado	Unite
2023	5	DXB/OMDB	Dubai International Airport	Garhoud, Dubai, Dubai	United Ar
2023	6	LAX/KLAX	Los Angeles International Airport	Los Angeles, California	Unite
2023	18	FRA/EDDF	Frankfurt Airport	Frankfurt, Hesse	Ger
2024	1	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	Unite
2024	2	DXB/OMDB	Dubai International Airport	Garhoud, Dubai, Dubai	United Ar

2024	3	DFW/KDFW	Dallas Fort Worth International Airport	Dallas–Fort Worth, Texas	Unite
2024	6	DEN/KDEN	Denver International Airport	Denver, Colorado	Unite
2024	8	LAX/KLAX	Los Angeles International Airport	Los Angeles, California	Unite
2024	16	FRA/EDDF	Frankfurt Airport	Frankfurt, Hesse	Ger
2025	1	ATL/KATL	Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	Unite
2025	2	DXB/OMDB	Dubai International Airport	Garhoud, Dubai, Dubai	United Ar
2025	3	DFW/KDFW	Dallas Fort Worth International Airport	Dallas–Fort Worth, Texas	Unite
2025	6	DEN/KDEN	Denver International Airport	Denver, Colorado	Unite
2025	11	LAX/KLAX	Los Angeles International Airport	Los Angeles, California	Unite
2025	21	FRA/EDDF	Frankfurt Airport	Frankfurt, Hesse	Ger

Table 1 presents the rank and total passenger counts for each of the selected airports from 2020 through 2025. Several notable patterns emerge from the data. Atlanta (ATL) consistently holds the top rank across every year in the dataset. The year 2020 stands out because every airport experienced a sharp decline in passenger traffic.

1.3 Passenger Traffic Visualization

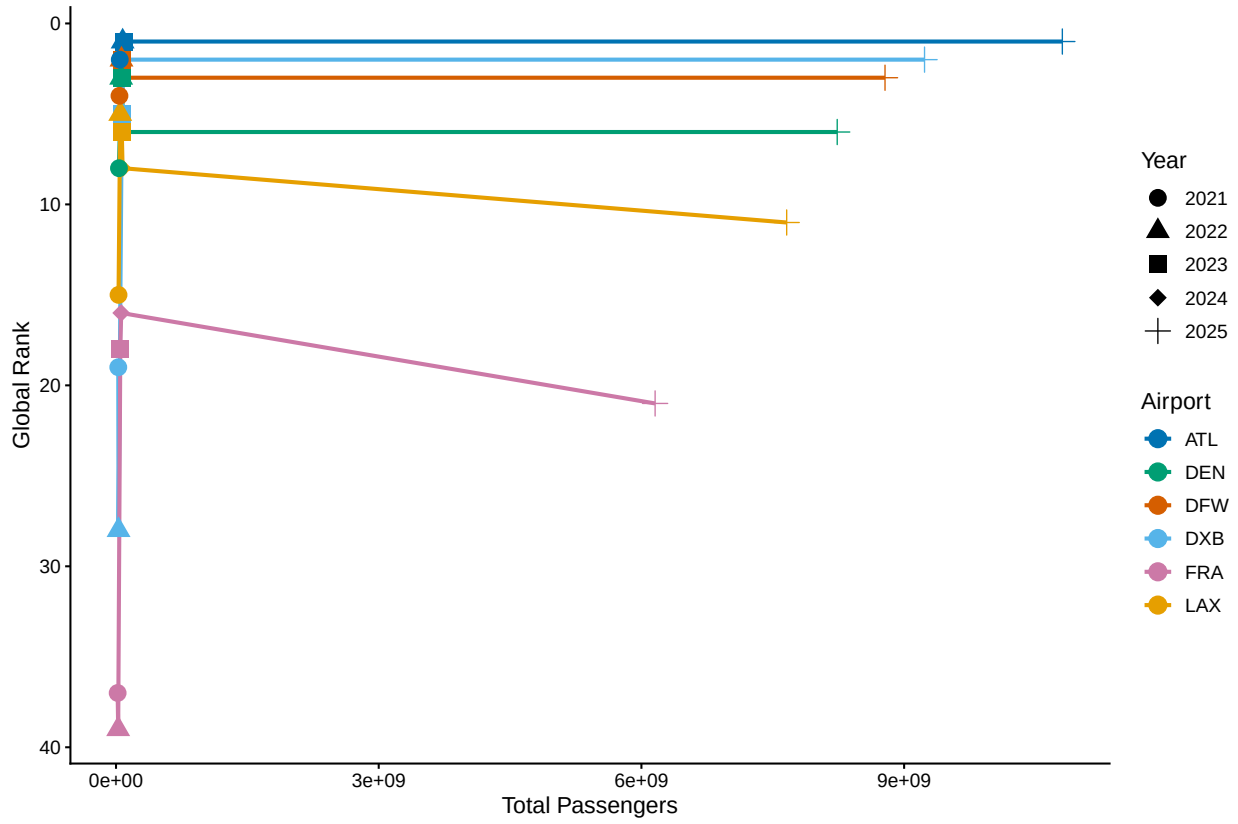


Figure 1: Total Passengers vs. Rank for Six Major International Airports (2020–2025)

Figure 1 visualizes the relationship between total annual passengers and global ranking position for each airport, with each observation year distinguished by a unique shape. This plot reveals the relationship between passenger volume and rank: airports with higher passenger counts occupy the numerically lower (better) rank positions at the top of the vertical axis. The inverted y-axis was chosen so that the most highly ranked airports appear at the top of the display.

Atlanta's trends are clear as a stand-out cluster in the upper-right corner of the plot. The 2020 data points for every airport are clustered toward the left side of the horizontal axis. As the years progress from 2021 through 2025, the points migrate rightward, illustrating the gradual growth. DXB's line, for example, follows a visibly different trajectory from the North American airports, confirming the observation from the table that Dubai's growth was faster and more pronounced.

2 Monte Carlo Numerical Integration

2.1 The Weibull Distribution Function

The function chosen for numerical integration via Monte Carlo simulation is the Weibull probability density function (PDF) with a shape parameter of 1.5 and a scale parameter of 1.

where $k = 1.5$ is the shape parameter and $\lambda = 1$ is the scale parameter. With these parameter values, with its peak occurring at approximately $x \approx 0.63$ and the bulk of its probability mass concentrated between $x = 0$ and $x = 2$. The function is defined for all $x \geq 0$ and approaches zero as x increases. Because the Weibull PDF integrates to exactly 1 over its entire support , the expected value of the integral from 0 to 4 should be very close to 1, since nearly all of the distribution's mass falls within this interval.

2.2 Monte Carlo Methodology

Monte Carlo numerical integration is a technique that estimates the value of a definite integral by exploiting random sampling.If we draw a large number of random points uniformly within a bounding rectangle around a region of interest, the proportion of those points that fall below the curve provide an estimate of the ratio of the area under the curve to the total area of the rectangle. By multiplying this proportion by the rectangle's area, we obtain an estimate of the integral.

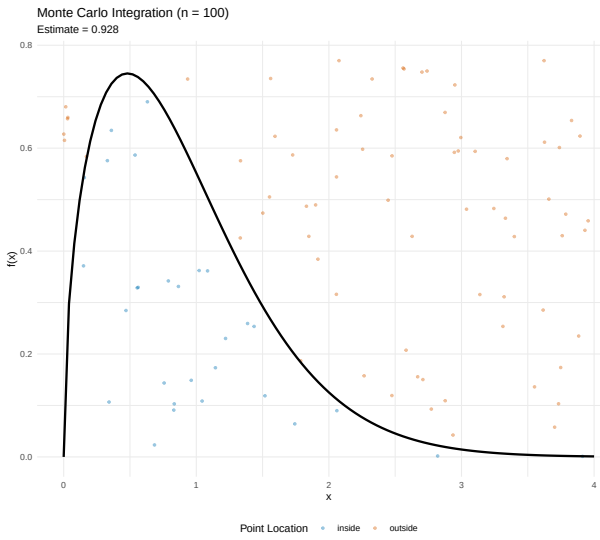
In this analysis, the bounding rectangle extends from $x = 0$ to $x = 4$ on the horizontal axis and from $y = 0$ to $y = 0.8$ on the vertical axis, yielding a total rectangle area of 3.2. For each random point (x_i, y_i) drawn within this rectangle, we evaluate $f(x_i)$ and determine whether $y_i \leq f(x_i)$. Points satisfying this condition lie within the area under the curve and are counted as "inside," while those above the curve are counted as "outside."

$A_{\text{rect}} = 3.2$ is the area of the bounding rectangle, n_{inside} is the number of points falling under the curve, and n_{total} is the total number of sampled points. The accuracy of this estimate improves as n_{total} increases, governed by the Central Limit Theorem, which guarantees that the sampling error decreases at a rate proportional to $1/\sqrt{n}$.

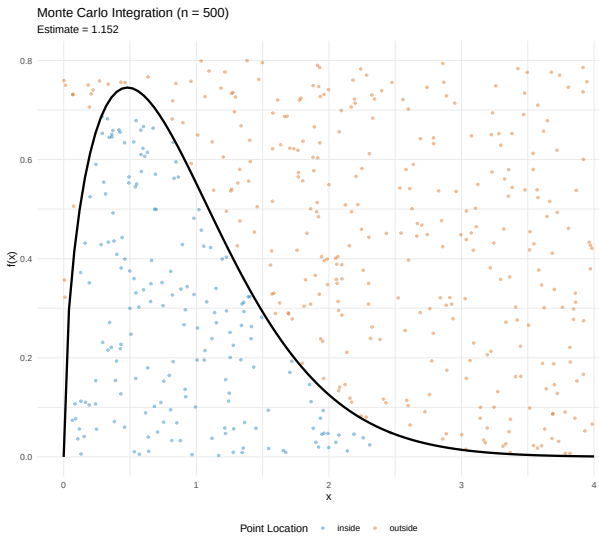
2.3 Results and Visualization

Figure 2 presents a small multiple visualization of the Monte Carlo integration process at four simulation resolutions: 100, 500, 2,000, and 10,000 randomly sampled points. Each sub-panel displays the Weibull PDF as a smooth black curve, overlaid with the randomly sampled points colored blue if they fall inside and orange if they fall outside (above the curve). The subtitle of each panel reports the corresponding integral estimate.

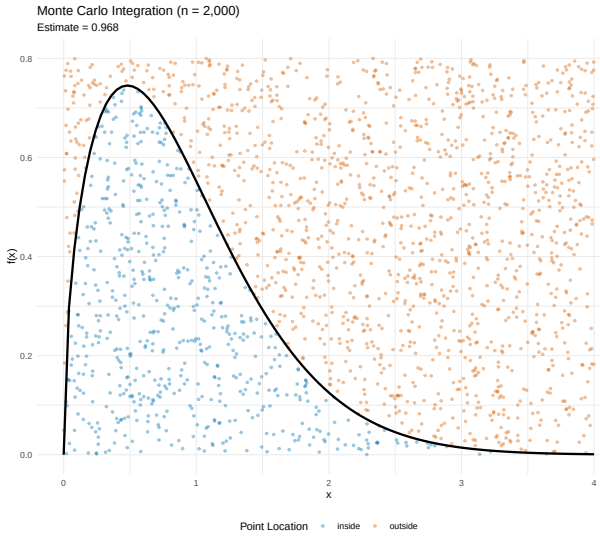
Several observations emerge from this visualization. At the lowest resolution ($n = 100$), the estimate is not accurate because of the lower sampling. As the number of points increases to 500, the estimate improves, At $n = 2,000$, the estimate becomes accurate, and resembles the shape of the Weibull PDF. Finally, at $n = 10,000$, the points produce an estimate that is very close to the true value.



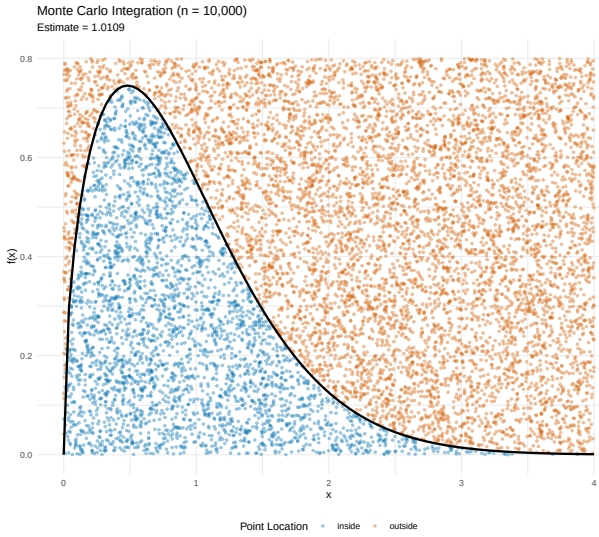
(a) n = 100 points



(b) n = 500 points



(c) n = 2{,}000 points



(d) n = 10{,}000 points

Figure 2: Monte Carlo Integration of the Weibull PDF (shape=1.5, scale=1) at Four Simulation Resolutions

3 Planning and Prompting GenAI Tools

3.1 Data Context and Tidying Plan

3.2 Data Context and Tidying Plan

The data for this section come from a study that followed 31 women over the course of four years. At the start of the study, researchers measured the amount of calcium in the ulna. Fifteen of the women received no special treatment and comprise the null treatment (control) group, while the remaining sixteen women received dietary support and followed a regular exercise program, comprising the diet/exercise treatment group. The raw dataset, `calcium.csv`, is structured with eight columns: the first four columns hold the control group's measurements (Initial, Year 1, Year 2, Year 3) and the last four columns hold the treatment group's measurements at the same time points. This format is not tidy.

Goal: Tidy the calcium table into a glyph-ready format and then create a line chart that shows the difference in ulnar calcium measurements over time between the treatment group and the null group.

Needs:

Nouns: RStudio, calcium data, a blank R script, calcium cleaned data, a line chart.

Verbs: Open RStudio, create a blank R script, load the data, clean the data, pivot the data, create a new column to identify treatment or null group, create a visualization.

Steps:

1. Open RStudio.
2. Create a blank R script and load the data from the calcium table.
3. Load any necessary packages for cleaning and creating visualizations (`tidyverse` and `ggplot2`).
4. Separate the two groups and create a new column assigning an ID to each row.
5. Use `pivot_longer()` to ensure that each row contains a single subject's measurement at a single time point.
6. Using `ggplot2`, build a line chart with time (Initial, Year 1, Year 2, Year 3) on the x-axis and the calcium values on the y-axis.
7. Assign a distinct color to the treatment group and a different color to the null group.
8. Create labels and a title for the visualization.
9. Run the code and ensure the output is as intended.
10. Draw conclusions regarding the effectiveness of the treatments based on the visualization.

3.3 Plan Informed-Response

In this subsection, the calcium dataset and the tidying plan developed above were prompted to Claude alongside the original data file. Claude's full response, including its analysis of the data and the generated R code, is presented below.

Tool Used: Claude

Prompt:

```
I have the attached data set which involves data drawn from 31 women over
the course of four years, the amount of calcium in the ulna for each woman
was measured, 15 did not change anything and the other 16 were in a treatment
group. I have created this plan for tidying the data and creating a
visualization, help me create the R code. Goal: Tidy the calcium table to a
glyph ready form and then create a line chart which shows the difference in
```

ulnar calcium measurements over time between the treatment group and the null group.

Needs:
Nouns: Rstudio, Calcium data, Blank Rscript, calcium cleaned data, a line chart
Verbs: Open Rstudio, create a blank rscript, load the data, clean the data, pivot the data, create new column to identify treatment or null, create a visualization

Steps:
1: Open RStudio
2: Create a blank RScript and load the data from the calcium table
3: Load any necessary packages for cleaning and creating visualizations (tidyverse and ggplot2)
4: Separate the two groups and create a new column and assigning an "ID" to each row
5: Use pivot_longer to ensure that each row contains a single subject
6: Using ggplot build a line chart with time (initial, year 1, year 2, year3) on the x-axis and the calcium values on the Y axis.
9: Assign a color to the treatment group and then a different color to the null group.
10: Create labels and a title for the visualization
11: Run the code and ensure the output is as intended.
12: Draw conclusions regarding the effectiveness of the treatments based on the visualization.

Response:

I can see the data clearly. The left 4 columns (y0-y3) are the control group (15 women, with a missing last row), and the right 4 columns (y0_1-y3_1) are the treatment group (16 women). Here's a walkthrough of the key decisions made in the script:

Data entry & structure. The image shows the control group in the left 4 columns (15 rows) and the treatment group in the right 4 columns (16 rows, with the asterisk row in the control side being blank). The data is entered as two separate data frames reflecting this split.

Tidying with pivot_longer. Each row becomes one observation per subject per time point --- so a subject with 4 measurements becomes 4 rows. The raw column names (y0-y3) are recoded to readable labels (Year 0-Year 3) and stored as an ordered factor so ggplot respects the correct time sequence on the x-axis.

Two-layer visualization. The chart draws individual subject lines at low opacity (so you can see variation within each group) with bold group mean lines on top. This is more informative than just plotting means alone.

Conclusions to look for once you run it:

If the Treatment line stays higher or declines less than the Control line from Year 0 -> Year 3, the treatment helped preserve ulnar calcium.
Check that Year 0 baselines are similar between groups --- if they're not,

```
it complicates interpretation.
A sharp Year 3 dip in both groups would suggest a time-based effect beyond
any treatment benefit.

# ============================================================
# Ulnar Calcium Study: Data Tidying & Visualization
# 31 women measured over 4 years (Year 0-3)
# Control group (n=15): y0, y1, y2, y3
# Treatment group (n=16): y0_1, y1_1, y2_1, y3_1
# ============================================================

# --- Step 1: Load packages ---
library(tidyverse)
library(ggplot2)

# --- Step 2: Enter the raw data ---

# Control group (15 women, no treatment)
control_raw <- data.frame(
  y0 = c(87.3, 59.0, 76.7, 70.6, 54.9, 78.2, 73.7, 61.8, 85.3, 82.3,
         68.6, 67.8, 66.2, 81.0, 72.3),
  y1 = c(86.9, 60.2, 76.5, 76.1, 55.1, 75.3, 70.8, 68.7, 84.4, 86.9,
         65.4, 69.2, 67.0, 82.3, 74.6),
  y2 = c(86.7, 60.0, 75.7, 72.1, 57.2, 69.1, 71.8, 68.2, 79.2, 79.4,
         72.3, 66.3, 67.0, 86.8, 75.3),
  y3 = c(75.5, 53.6, 69.5, 65.3, 49.0, 67.6, 74.6, 57.4, 67.0, 77.4,
         60.8, 57.9, 56.2, 73.9, 66.1)
)

# Treatment group (16 women)
treatment_raw <- data.frame(
  y0 = c(83.8, 65.3, 81.2, 75.4, 55.3, 70.3, 76.5, 66.0, 76.7, 77.2,
         67.3, 50.3, 57.7, 74.3, 74.0, 57.3),
  y1 = c(85.5, 66.9, 79.5, 76.7, 58.3, 72.3, 79.9, 70.9, 79.0, 74.0,
         70.7, 51.4, 57.0, 77.7, 74.7, 56.0),
  y2 = c(86.2, 67.0, 84.5, 74.3, 59.1, 70.6, 80.4, 70.3, 76.9, 77.8,
         68.9, 53.6, 57.5, 72.6, 74.5, 64.7),
  y3 = c(81.2, 60.6, 75.2, 66.7, 54.2, 68.6, 71.6, 64.1, 70.3, 67.9,
         65.9, 48.0, 51.5, 68.0, 65.7, 53.0)
)

# --- Step 3: Add group labels and subject IDs ---
control_raw <- control_raw %>%
  mutate(
    ID    = paste0("C", row_number()),
    group = "Control"
  )

treatment_raw <- treatment_raw %>%
  mutate(
    ID    = paste0("T", row_number()),
    group = "Treatment"
  )
```

```
# --- Step 4: Combine and pivot to long (tidy) format ---
calcium_long <- bind_rows(control_raw, treatment_raw) %>%
  pivot_longer(
    cols      = c(y0, y1, y2, y3),
    names_to  = "time_raw",
    values_to = "calcium"
  ) %>%
  mutate(
    time = recode(time_raw,
                  "y0" = "Year 0",
                  "y1" = "Year 1",
                  "y2" = "Year 2",
                  "y3" = "Year 3"),
    time = factor(time, levels = c("Year 0", "Year 1", "Year 2",
                                   "Year 3"))
  )

# --- Step 5: Compute group means per time point ---
calcium_summary <- calcium_long %>%
  group_by(group, time) %>%
  summarise(mean_calcium = mean(calcium), .groups = "drop")

# --- Step 6: Build the visualization ---
ggplot() +
  geom_line(
    data    = calcium_long,
    mapping = aes(x = time, y = calcium, group = ID, color = group),
    alpha   = 0.20,
    linewidth = 0.4
  ) +
  geom_line(
    data    = calcium_summary,
    mapping = aes(x = time, y = mean_calcium, group = group, color = group),
    linewidth = 1.4
  ) +
  geom_point(
    data    = calcium_summary,
    mapping = aes(x = time, y = mean_calcium, color = group),
    size    = 3
  ) +
  scale_color_manual(
    values = c("Control" = "#E07A5F", "Treatment" = "#3D9B8F"),
    name   = "Group"
  ) +
  labs(
    title    = "Ulnar Calcium Measurements Over Time",
    subtitle = "Bold lines show group means; faint lines show individual subjects",
    x        = "Time Point",
    y        = "Ulnar Calcium (mg)",
    caption  = "Control n=15 | Treatment n=16"
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold", size = 15),
```

```
    plot.subtitle = element_text(color = "gray40", size = 10),
    legend.position = "top",
    panel.grid.minor = element_blank()
  )

ggsave("calcium_lineplot.png", width = 8, height = 5.5, dpi = 150)
```

3.4 Generic Response

In contrast to the plan-informed approach above, this subsection presents the results obtained when the same AI tool was given a generic, unstructured prompt alongside the same calcium dataset.

Tool Used: Claude

Prompt:

```
Here is a data set that I need you to use R to clean and make a visualization
showing the trends of the responses over time
```

Response:

```
Looking at your data, I can see 8 variables (y0-y3 and y0_1-y3_1) across 14
rows, with some missing values (*) in the last row. Here's clean,
well-commented R code to wrangle and visualize it:

Here's what the script does, step by step:

Data wrangling:

All values are entered manually (since the * cells are missing, they're
treated as NA and dropped with filter(!is.na(value)))
A time index is added per row, then the data is pivoted from wide to long
format
Two derived columns are created: group (Group 1 = y0-y3, Group 2 = y0_1-y3_1)
and timepoint (the 0-3 suffix, representing the within-group time progression)

Visualization:

A summary table of mean +/- SE is computed per group x timepoint
The plot layers: raw jittered points -> +/-1 SE ribbon -> trend line -> mean
markers
Run ggsave(...) at the end to export --- or swap it for print(p) if working
interactively

# -- Load libraries --
library(ggplot2)
library(tidyr)
library(dplyr)

# -- 1. Enter the data --
raw <- data.frame(
  y0   = c(87.3, 59.0, 76.7, 70.6, 54.9, 78.2, 73.7, 61.8, 85.3, 82.3,
           68.6, 67.8, 66.2, 81.0, 72.3),
```

```
  y1   = c(86.9, 60.2, 76.5, 76.1, 55.1, 75.3, 70.8, 68.7, 84.4, 86.9,
           65.4, 69.2, 67.0, 82.3, 74.6),
  y2   = c(86.7, 60.0, 75.7, 72.1, 57.2, 69.1, 71.8, 68.2, 79.2, 79.4,
           72.3, 66.3, 67.0, 86.8, 75.3),
  y3   = c(75.5, 53.6, 69.5, 65.3, 49.0, 67.6, 74.6, 57.4, 67.0, 77.4,
           60.8, 57.9, 56.2, 73.9, 66.1),
  y0_1 = c(83.8, 65.3, 81.2, 75.4, 55.3, 70.3, 76.5, 66.0, 76.7, 77.2,
           67.3, 50.3, 57.7, 74.3, 74.0),
  y1_1 = c(85.5, 66.9, 79.5, 76.7, 58.3, 72.3, 79.9, 70.9, 79.0, 74.0,
           70.7, 51.4, 57.0, 77.7, 74.7),
  y2_1 = c(86.2, 67.0, 84.5, 74.3, 59.1, 70.6, 80.4, 70.3, 76.9, 77.8,
           68.9, 53.6, 57.5, 72.6, 74.5),
  y3_1 = c(81.2, 60.6, 75.2, 66.7, 54.2, 68.6, 71.6, 64.1, 70.3, 67.9,
           65.9, 48.0, 51.5, 68.0, 65.7)
)

# -- 2. Add a time index (row = observation point) --
raw$time <- seq_len(nrow(raw))

# -- 3. Pivot to long format --
long <- raw %>%
  pivot_longer(
    cols      = -time,
    names_to  = "variable",
    values_to = "value"
  ) %>%
  filter(!is.na(value)) %>%
  mutate(
    group     = if_else(grepl("_1$", variable), "Group 2", "Group 1"),
    timepoint = as.integer(sub(".*y(\\d).*", "\\1", variable))
  )

# -- 4. Compute per-variable summary (mean +/- SE across observations) --
summary_df <- long %>%
  group_by(group, timepoint) %>%
  summarise(
    mean_val = mean(value, na.rm = TRUE),
    se       = sd(value, na.rm = TRUE) / sqrt(n()),
    .groups  = "drop"
  )

# -- 5. Plot --
p <- ggplot(summary_df, aes(x = timepoint, y = mean_val,
                            color = group, fill = group)) +
  geom_ribbon(aes(ymin = mean_val - se, ymax = mean_val + se),
              alpha = 0.15, color = NA) +
  geom_line(linewidth = 1.1) +
  geom_jitter(data = long,
              aes(x = timepoint, y = value, color = group),
              width = 0.07, alpha = 0.25, size = 1.6, inherit.aes = FALSE) +
  geom_point(size = 3.5, shape = 21, color = "white", stroke = 1.5) +
  scale_x_continuous(breaks = 0:3,
                     labels = paste0("T", 0:3)) +
  scale_color_manual(values = c("Group 1" = "#2C7BB6", "Group 2" = "#D7191C")) +
```

```
  scale_fill_manual( values = c("Group 1" = "#2C7BB6", "Group 2" = "#D7191C")) +
  labs(
    title    = "Response Values Over Time",
    subtitle = "Mean +/- SE with individual observations",
    x        = "Time Point",
    y        = "Response Value",
    color    = NULL,
    fill     = NULL
  ) +
  theme_minimal(base_size = 13) +
  theme(
    plot.title       = element_text(face = "bold", size = 15),
    plot.subtitle    = element_text(color = "grey45", margin = margin(b = 10)),
    legend.position  = "top",
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "grey90")
  )

ggsave("plot.png", plot = p, width = 7, height = 5, dpi = 150)
cat("Plot saved to plot.png\n")
```

3.5 Comparative Analysis

The difference between the plan-informed response and the generic response reveals the mistakes that are possible if you neglect to create a plan for yourself or for any generative artificial intelligence that is helping you to create code.

The first difference that I noticed between the two was Claude's misunderstanding of the data structure. Claude described the dataset as having "8 variables across 14 rows" and treated each row as a time point rather than as an individual subject. This led to a some issues down the line once it created the code the data was pivoted incorrectly. The groups were labeled as "Group 1" and "Group 2" rather than "Control" and "Treatment," no subject IDs were created, and the visualization showed time on the x-axis as a numeric scale (0–3) without labels. Claude entirely dropped the treatment groups 16th data because there was not a 16th women in the control group.

The plan-informed response however created a useful code. It followed the steps without much deviation and was able to make a better visualization that succesfully reached my initial goal which I provided to it at the beginning.

4 Reflection

Coming into this course I have had experience programming in the languages: Python, SQL, and some Java and HTML, but I had not had the opportunity to explore R so this has been very exciting learning a new language throughout the course. Previously, I had not practiced writing out explicit plans involving goals, needs or steps and I believe that this PCIP process will prove to be very beneficial as I continue working on code throughout my education and future career. Most of the previous courses that I've taken where we program we did not dive deep into many of the useful packages that the languages supported and being able to get familiar with installing, loading and exploring packages has also been a very useful experience. The data that we worked with for the homework assignments have all been fun and interesting cases which made completing assignments very enjoyable. Overall, I have thoroughly enjoyed STAT 184, the lectures, content and assignments have all felt meaningful, useful, and kept me engaged through the interesting data and fun activites. At the beginning of the course I had set my own personal goal of working to the best of my ability to learn and understand all material through giving my best efforts on all activities and readings and make

a strong effort to earn the highest grade possible. I believe I have achieved that goal, I am confident in my understanding of all of the material we have covered throughout the course and anticipate receiving a grade that I am pleased with as a result of applying what I learned here into the assignments as well as in my participation and success at the datafest event. I'd like to thank Dr. Bubb and the teaching team for a great semester!

5 Appendix: Code Appendix

All code used to produce the analyses and visualizations in this document is below, organized by section.

5.1 Busiest Airports: Data Scraping and Cleaning

```
# ---- Load Required Packages ----
library(tidyverse)   # Data manipulation and visualization
library(rvest)       # Web scraping
library(janitor)     # Clean column names
library(knitr)       # Table formatting
library(kableExtra)  # Styled tables
library(ggplot2)     # Grammar of graphics

# ---- Step 1: Scrape Wikipedia Page for Airport Tables ----
# The Wikipedia page contains multiple tables; tables [[2]] through [[7]]
# correspond to the years 2025 through 2020.
airportRawList <- read_html(
  x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |>
  html_elements(css = "table") |>
  html_table()

# ---- Step 2: Extract Each Year's Table ----
# Apply clean_names() for consistent column naming and add a year identifier.
year_2025 <- airportRawList[[2]] |> clean_names() |> mutate(year = 2025)
year_2024 <- airportRawList[[3]] |> clean_names() |> mutate(year = 2024)
year_2023 <- airportRawList[[4]] |> clean_names() |> mutate(year = 2023)
year_2022 <- airportRawList[[5]] |> clean_names() |> mutate(year = 2022)
year_2021 <- airportRawList[[6]] |> clean_names() |> mutate(year = 2021)
year_2020 <- airportRawList[[7]] |> clean_names() |> mutate(year = 2020)

# ---- Step 3: Combine All Years into One Data Frame ----
all_airports <- bind_rows(year_2025, year_2024, year_2023,
                          year_2022, year_2021, year_2020)

# ---- Step 4: Clean and Filter ----
airports_clean <- all_airports |>
  mutate(
    # Remove reference annotations (e.g., "[1]") from passenger counts
    totalpassengers = str_remove_all(totalpassengers, "[^0-9]"),
    # Convert passenger counts from character to numeric
    totalpassengers = as.numeric(totalpassengers),
    # Replace NA rank changes with 0 (no change)
    rankchange = replace_na(rankchange, 0)
```

```r
  ) |>
  # Select and reorder columns for clarity
  select(
    year, rank, code_iata_icao, airport, location,
    country, totalpassengers, rankchange
  ) |>
  # Retain only the six airports of interest
  filter(code_iata_icao %in% c(
    "ATL/KATL", "FRA/EDDF", "DXB/OMDB",
    "LAX/KLAX", "DFW/KDFW", "DEN/KDEN"
  ))
```

5.2 Busiest Airports: Table Generation

```r
# ---- Build a Formatted Table ----
airports_clean |>
  arrange(year, rank) |>
  mutate(totalpassengers = format(totalpassengers, big.mark = ",")) |>
  kable(
    col.names = c(
      "Year", "Rank", "Code", "Airport", "Location",
      "Country", "Total Passengers", "Rank Change"
    ),
    align = "c"
  ) |>
  kable_styling(
    latex_options = c("striped", "hold_position", "scale_down"),
    full_width = FALSE
  )
```

5.3 Busiest Airports: Plot Generation

```r
# ---- Build the Scatter-Line Plot ----
ggplot(airports_clean, aes(
  x = totalpassengers,
  y = rank,
  color = code_iata_icao,
  group = code_iata_icao,
  shape = factor(year)
)) +
  geom_line(linewidth = 1) +          # Connect points by airport
  geom_point(size = 4) +              # Mark each year's observation
  scale_y_reverse() +                 # Rank 1 at the top
  scale_color_manual(
    values = c(
      "ATL/KATL"  = "#0072B2",
      "DFW/KDFW"  = "#D55E00",
      "DEN/KDEN"  = "#009E73",
      "LAX/KLAX"  = "#E69F00",
```

```r
      "FRA/EDDF"  = "#CC79A7",
      "DXB/OMDB"  = "#56B4E9"
    ),
    labels = c(
      "ATL/KATL"  = "ATL",
      "DFW/KDFW"  = "DFW",
      "DEN/KDEN"  = "DEN",
      "LAX/KLAX"  = "LAX",
      "FRA/EDDF"  = "FRA",
      "DXB/OMDB"  = "DXB"
    )
  ) +
  scale_shape_manual(
    values = c(16, 17, 15, 18, 3, 7),
    name = "Year"
  ) +
  labs(
    x = "Total Passengers",
    y = "Global Rank",
    color = "Airport",
    shape = "Year"
  ) +
  theme_classic() +
  theme(
    text = element_text(size = 12),
    legend.position = "right",
    plot.title = element_text(face = "bold")
  )
```

5.4 Monte Carlo Integration: Function Definition

```r
# ---- Define the Weibull PDF (shape = 1.5, scale = 1) ----
f <- function(x) {
  dweibull(x, shape = 1.5, scale = 1)
}

# ---- Monte Carlo Simulation Function ----
# Simulates n random points in the bounding rectangle [x_min, x_max] x [y_min, y_max]
# and estimates the integral of func over [x_min, x_max].
#
# Parameters:
#   n      - Number of random points to sample
#   func   - The function to integrate
#   x_min  - Lower bound of integration
#   x_max  - Upper bound of integration
#   y_min  - Lower bound of the sampling rectangle
#   y_max  - Upper bound of the sampling rectangle
#
# Returns: A list with $df (data frame of points) and $estimate (integral estimate)
monte_carlo_sim <- function(n, func, x_min, x_max, y_min, y_max) {
  # Generate uniform random points
```

```r
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  # Evaluate the function at each x
  f_x <- func(x_vals)

  # Classify each point as inside (under curve) or outside (above curve)
  df <- data.frame(
    x   = x_vals,
    y   = y_vals,
    f_x = f_x
  ) |>
    mutate(flag = ifelse(y <= f_x, "inside", "outside"))

  # Compute the integral estimate
  area       <- (x_max - x_min) * (y_max - y_min)
  proportion <- mean(y_vals <= f_x)
  estimate   <- area * proportion

  return(list(df = df, estimate = estimate))
}
```

5.5 Monte Carlo Integration: Simulation and Visualization

```r
# ---- Helper: Build a Single Monte Carlo Plot ----
build_mc_plot <- function(result, n_label) {
  ggplot(result$df, aes(x = x, y = y)) +
    geom_point(aes(color = flag), alpha = 0.4, size = 0.8) +
    stat_function(
      fun = dweibull,
      args = list(shape = 1.5, scale = 1),
      xlim = c(0, 4),
      color = "black",
      linewidth = 1
    ) +
    scale_color_manual(
      values = c("inside" = "#0072B2", "outside" = "#D55E00")
    ) +
    labs(
      x = "x",
      y = "f(x)",
      color = "Point Location",
      title = paste0("Monte Carlo Integration (", n_label, ")"),
      subtitle = paste("Estimate =", round(result$estimate, 4))
    ) +
    theme_minimal(base_size = 10) +
    theme(legend.position = "bottom")
}

# ---- Run Simulations at Four Resolutions ----
set.seed(42)  # For reproducibility
```

```r
res_100   <- monte_carlo_sim(100,    f, 0, 4, 0, 0.8)
res_500   <- monte_carlo_sim(500,    f, 0, 4, 0, 0.8)
res_2000  <- monte_carlo_sim(2000,   f, 0, 4, 0, 0.8)
res_10000 <- monte_carlo_sim(10000,  f, 0, 4, 0, 0.8)

# ---- Display the Four-Pane Small Multiple ----
# In the rendered document, these four plots appear as a 2x2 grid
# using Quarto's layout-ncol and fig-subcap options.
build_mc_plot(res_100,   "n = 100")
build_mc_plot(res_500,   "n = 500")
build_mc_plot(res_2000,  "n = 2,000")
build_mc_plot(res_10000, "n = 10,000")
```